



UNIVERSIDADE VEIGA DE ALMEIDA
CURSO DE SISTEMAS DE INFORMAÇÃO

GABRIEL GARCIA GONÇALVES DE CARVALHO – 1250202190

VINICIUS LUIS DA SILVA – 1250109403

JOÃO VITOR TOSTES DE SOUZA – 1250106169

GUILHERME SILVA TIMAS SOARES – 1250111076

ATLAS JORNAL

Rio de Janeiro - RJ

2026

GABRIEL GARCIA GONÇALVES DE CARVALHO

VINICIUS LUIS DA SILVA

JOÃO VITOR TOSTES DE SOUZA

GUILHERME SILVA TIMAS SOARES

ATLAS JORNAL

Documentação textual apresentada como requisito de aprovação na disciplina de Aplicações na Internet, ministrada pelo docente Eduardo Pareto na Universidade Veiga de Almeida.

Professor: Eduardo Luiz Pareto

Rio de Janeiro - RJ

2026

RESUMO

O Atlas Jornal é um sistema web de publicação de notícias desenvolvido na disciplina de Aplicações na Internet, no 5º semestre do curso de Sistemas de Informação da Universidade Veiga de Almeida. O projeto foi construído sem o uso de frameworks, utilizando exclusivamente PHP puro, MySQL, HTML5, CSS3 e JavaScript, com o objetivo de demonstrar o domínio dos fundamentos do desenvolvimento web. O sistema é composto por duas frentes: um site público de visualização de notícias, organizadas por categoria e seção, e uma área administrativa restrita com controle de sessão e perfis de acesso, na qual redatores criam conteúdo, revisores analisam e aprovam publicações, e administradores gerenciam usuários e permissões. A arquitetura adota o padrão MVC implementado manualmente, com persistência de dados via PDO e autenticação baseada em sessões PHP nativas.

Palavras-chave: desenvolvimento web; PHP; MySQL; MVC; sistema de notícias; autenticação; controle de acesso; persistência de dados.

SUMÁRIO

1 INTRODUÇÃO	5
2 DESCRIÇÃO DO SISTEMA	5
2.1 Objetivos	5
2.2 Tecnologias Utilizadas	6
2.3 Escopo Funcional e Fluxo de Operações	6
3 MODELAGEM DO BANCO DE DADOS	7
3.1 Diagrama Entidade-Relacionamento (DER)	7
3.2 Modelo Lógico	8
4 ARQUITETURA E EXPLICAÇÃO DO CÓDIGO	9
4.1 Padrão MVC — App.php e Auth.php	9
4.2 Banco de Dados — Database.php e schema.sql	10
4.3 Autenticação — AuthController.php	15
4.4 Notícias — Noticia.php, NoticiaRepository.php e NoticiaController.php	16
4.5 Revisão Editorial — RevisaoController.php	22
4.6 Solicitação de Cargo — SolicitacaoController.php	25
5 CONSIDERAÇÕES FINAIS	26
REFERÊNCIAS	28

1 INTRODUÇÃO

O Atlas Jornal é um sistema web desenvolvido na disciplina de Aplicações na Internet com o objetivo de simular o funcionamento de um portal de notícias digital completo. O projeto abrange desde o cadastro de usuários e o controle de acesso por perfis até a publicação e revisão de conteúdo jornalístico, implementando um fluxo editorial real, no qual o conteúdo passa por etapas de redação, análise e aprovação antes de ser exibido ao público.

A aplicação foi construída com PHP puro, sem o uso de frameworks externos, aplicando o padrão arquitetural MVC (Model-View-Controller) de forma manual, o que permitiu aos desenvolvedores compreender em profundidade o funcionamento de cada camada do sistema. O banco de dados utilizado é o MySQL, acessado via PDO com prepared statements para garantir segurança contra ataques de SQL injection.

2 DESCRIÇÃO DO SISTEMA

2.1 Objetivos

O sistema tem como propósito oferecer uma plataforma completa para a gestão de conteúdo jornalístico, com controle de acesso baseado em perfis de usuário. Os principais objetivos são: permitir que leitores visualizem notícias publicadas por categoria e seção; possibilitar que redatores criem, editem e enviem notícias para análise; oferecer a revisores ferramentas para aprovar, rejeitar ou arquivar conteúdos; e fornecer ao administrador o controle total sobre usuários, permissões e solicitações de cargo.

O sistema contempla também funcionalidades como busca de notícias, páginas institucionais, perfil de usuário com foto e assinatura, e um dashboard com visão geral das atividades recentes.

2.2 Tecnologias Utilizadas

Para o desenvolvimento do Atlas Jornal foram utilizadas as seguintes tecnologias: PHP 8 no back-end, com arquitetura MVC implementada manualmente; MySQL como sistema gerenciador de banco de dados relacional; PDO (PHP Data Objects) para conexão segura com o banco de dados; HTML5 e CSS3 para a construção das views; JavaScript para interações no lado do cliente; e Apache como servidor web, configurado via XAMPP para o ambiente de desenvolvimento local. O controle de acesso e autenticação é gerenciado por sessões PHP nativas.

2.3 Escopo Funcional e Fluxo de Operações

Para cumprir os objetivos propostos do portal *Atlas Jornal*, o sistema divide-se em frentes operacional e administrativa, mapeando as seguintes ações executadas por cada perfil de usuário:

- **Leitor (Usuário Não Autenticado):** Navega pela página principal, acessa e lê notícias categorizadas por seções (como *Hero*, Nacional e Internacional), utiliza a barra de busca textual e visualiza as páginas institucionais (Código de Ética, Termos de Uso e Contato).
- **Redator:** Após o login, ganha acesso ao painel de criação onde pode conceber novas matérias preenchendo título, resumo, conteúdo, seções e categorias. Possui autonomia para editar seus rascunhos salvos, excluir publicações que ainda não foram avaliadas ou reenviar para análise matérias que foram rejeitadas pelo corpo de revisores.
- **Revisor:** Visualiza o painel editorial contendo todas as notícias com o status EM_ANALISE. O revisor é encarregado de ler o material técnico e decidir entre: aprovar (o que altera o status da notícia para APROVADA e a publica na interface

principal), rejeitar (retornando-a ao redator com observações) ou arquivar conteúdos obsoletos.

- **Administrador:** Possui o nível mais alto de privilégio na plataforma. Gerencia o cadastro geral de usuários, analisa e responde a solicitações de cargo (promovendo leitores a redatores ou revisores) e tem poder moderador para excluir usuários ou notícias em desconformidade com a política do portal.

3 MODELAGEM DO BANCO DE DADOS

O banco de dados do sistema é denominado `jornal_atlasdb` e é composto por quatro tabelas principais: `Usuario`, `Noticia`, `Revisao` e `SolicitacaoCargo`. Cada tabela possui chave primária própria com auto incremento e as relações entre elas são estabelecidas por meio de chaves estrangeiras, garantindo a integridade referencial dos dados.

3.1 Diagrama Entidade-Relacionamento (DER)

O Diagrama Entidade-Relacionamento (DER) apresentado na Figura 1 representa o modelo conceitual do banco de dados do Atlas Jornal. Nele, a entidade `Usuario` é central no sistema, relacionando-se com as demais entidades por meio de papéis distintos: como `Redator`, que redige notícias; como `Revisor`, que realiza revisões; como `Solicitante`, que solicita cargos; e como `Administrador`, que avalia as solicitações. A cardinalidade das relações indica que um usuário pode redigir zero ou várias notícias, enquanto cada notícia pertence a exatamente um redator.

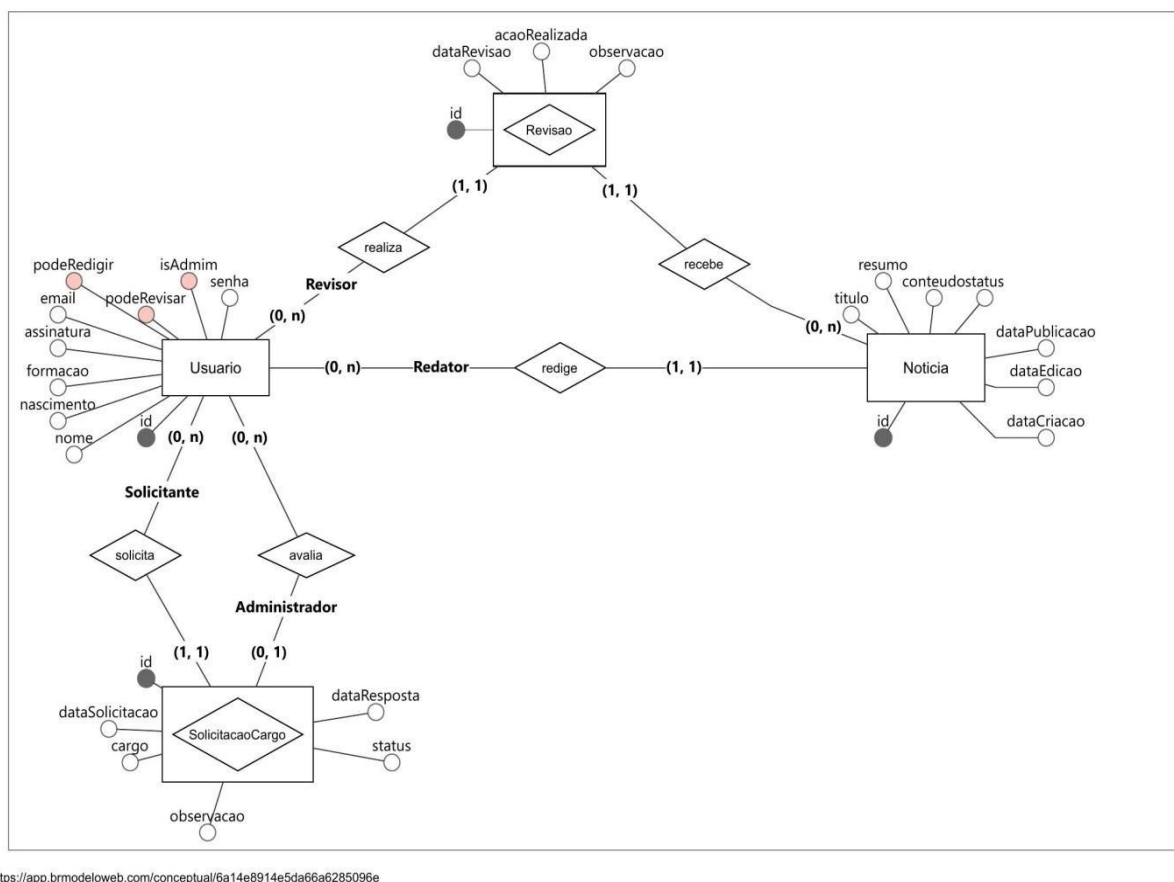


Figura 1 – Diagrama Entidade-Relacionamento do Atlas Jornal

Fonte: elaborado pelos autores (2026).

3.2 Modelo Lógico

O modelo lógico apresentado na Figura 2 detalha a estrutura física das tabelas, com os tipos de dados de cada atributo e a identificação das chaves primárias (PK) e estrangeiras (FK). A tabela *Usuario* armazena dados pessoais e flags booleanas de permissão. A tabela *Noticia* referencia o redator por chave estrangeira e utiliza campos ENUM para controlar o status e a seção da matéria. A tabela *Revisao* registra cada ação editorial realizada por um revisor. A tabela *SolicitacaoCargo* controla os pedidos de permissão feitos por usuários ao administrador.

26/05/2026, 11:17

Logic model - BRMW

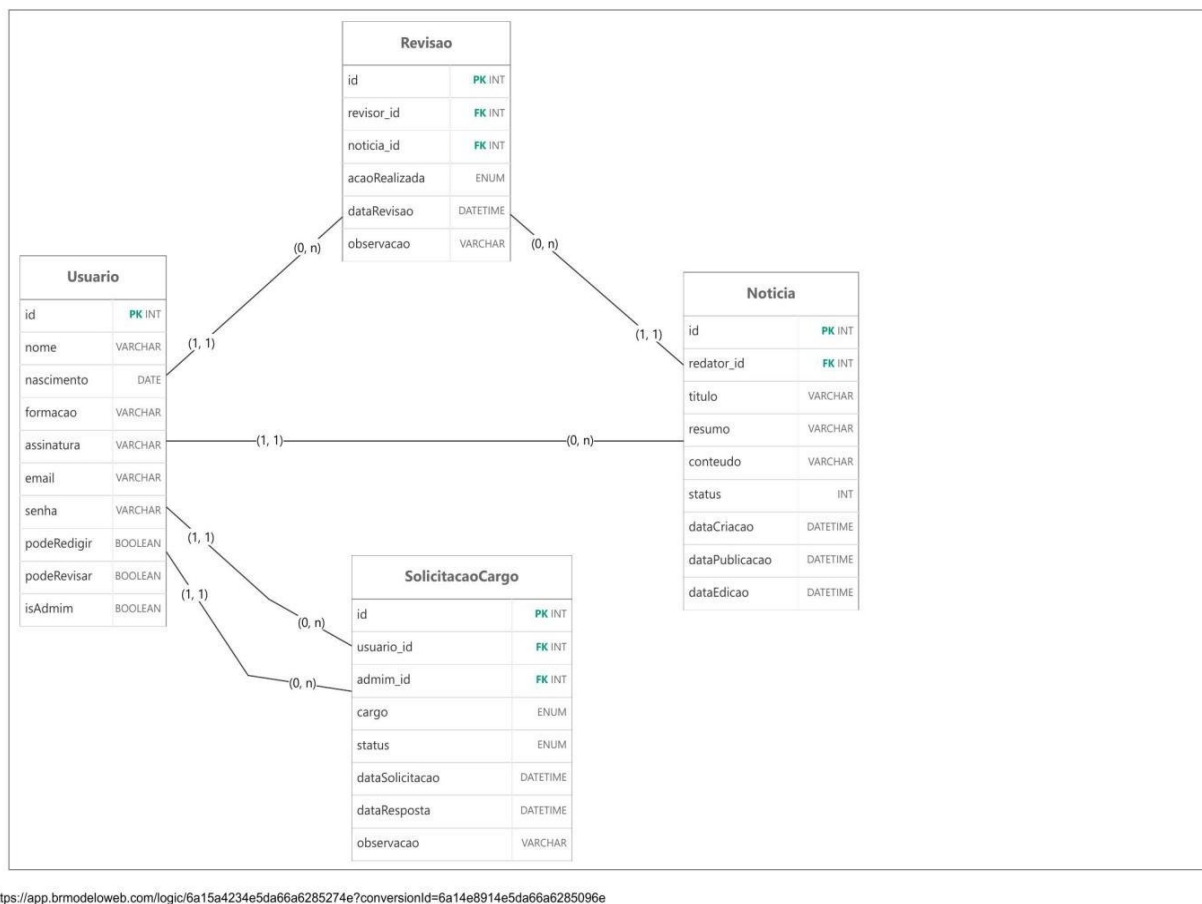


Figura 2 – Modelo Lógico do Banco de Dados

Fonte: elaborado pelos autores (2026).

4 ARQUITETURA E EXPLICAÇÃO DO CÓDIGO

O sistema segue o padrão MVC implementado manualmente em PHP puro, organizado em quatro camadas: Core (infraestrutura e roteamento), Models (entidades de domínio), Repositories (acesso ao banco de dados via PDO) e Controllers (lógica de negócio). As Views, localizadas em `app/Views/`, combinam HTML e PHP para renderizar as páginas ao usuário. A seguir são apresentados e explicados os principais arquivos de código do projeto.

4.1 Padrão MVC — App.php e Auth.php

O arquivo App.php é o roteador central do sistema. Ele recebe todas as requisições via URL, extrai o primeiro segmento do caminho e utiliza um switch/case para instanciar o controller correto. Além do roteamento, o método run() também atualiza os dados da sessão a cada requisição, garantindo que as informações do usuário logado (nome, cargo, permissões) estejam sempre atualizadas a partir do banco de dados.

Código 1 — app/Core/App.php

```
<?php
namespace App\Core;

use App\Controllers\AdminNoticiaController;
use App\Controllers\AuthController;
use App\Controllers\CategoryController;
use App\Controllers\DashboardController;
use App\Controllers\HomeController;
use App\Controllers\NoticiaController;
use App\Controllers\ProfileController;
use App\Controllers\RevisaoController;
use App\Controllers\SolicitacaoController;
use App\Controllers\UsuarioController;
use App\Repositories\UsuarioRepository;

class App
{
    public function run(): void
    {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }

        if (isset($_SESSION['usuario_logado']) && $_SESSION['usuario_logado']
=== true
        && isset($_SESSION['usuario_id'])) {
            $user = UsuarioRepository::find((int) $_SESSION['usuario_id']);
            if ($user) {
                $_SESSION['usuario_nome']      = $user->getNome();
                $_SESSION['usuario_email']     = $user->getEmail();
                $_SESSION['usuario_cargo']     = $user->getCargo();
                $_SESSION['usuario_foto']      = $user->getFoto();
                $_SESSION['usuario_podeRedigir'] = $user->getPodeRedigir();
                $_SESSION['usuario_podeRevisar'] = $user->getPodeRevisar();
                $_SESSION['usuario_isAdmin']    = $user->getIsAdmin();
            }
        }

        $url      = $_GET['url'] ?? '';
        $url      = trim($url, '/');
        $segments = $url === '' ? [] : explode('/', $url);
        $route    = $segments[0] ?? 'home';

        switch ($route) {
            case '': case 'home':
                (new HomeController())->index(); break;
            case 'sobre':
```

```

        (new HomeController())->sobre(); break;
    case 'categoria':
        (new CategoryController())->index($segments[1] ?? ''); break;
    case 'busca':
        (new HomeController())->busca(); break;
    case 'noticia':
        $ctrl = new NoticiaController();
        if (($segments[1] ?? '') === 'nova') {
            $_SERVER['REQUEST_METHOD'] === 'POST'
                ? $ctrl->store() : $ctrl->create();
        } elseif (($segments[1] ?? '') === 'minhas') {
            $ctrl->minhas();
        } elseif (($segments[2] ?? '') === 'editar' &&
is_numeric($segments[1] ?? '')) {
            $_SERVER['REQUEST_METHOD'] === 'POST'
                ? $ctrl->update($segments[1]) : $ctrl->
>edit($segments[1]);
        }
        break;
    case 'login':
        $ctrl = new AuthController();
        $_SERVER['REQUEST_METHOD'] === 'POST'
            ? $ctrl->authenticate() : $ctrl->login();
        break;
    case 'cadastro':
        $ctrl = new AuthController();
        $_SERVER['REQUEST_METHOD'] === 'POST'
            ? $ctrl->register() : $ctrl->cadastro();
        break;
    case 'logout':
        (new AuthController())->logout(); break;
    case 'revisao':
        $ctrl = new RevisaoController();
        if (($segments[1] ?? '') === 'aprovar' && ($segments[2] ?? ''))
            $ctrl->aprovar($segments[2]);
        elseif (($segments[1] ?? '') === 'rejeitar' && ($segments[2] ?? ''))
            $ctrl->rejeitar($segments[2]);
        else $ctrl->index();
        break;
    case 'solicitacoes':
        $ctrl = new SolicitacaoController();
        if (($segments[1] ?? '') === 'aprovar' && ($segments[2] ?? ''))
            $ctrl->aprovar($segments[2]);
        elseif (($segments[1] ?? '') === 'rejeitar' && ($segments[2] ?? ''))
            $ctrl->rejeitar($segments[2]);
        else $ctrl->index();
        break;
    case 'dashboard':
        (new DashboardController())->index(); break;
    case 'perfil':
        $ctrl = new ProfileController();
        $_SERVER['REQUEST_METHOD'] === 'POST'
            ? $ctrl->update() : $ctrl->index();
        break;
    default:
        http_response_code(404);
        require_once __DIR__ . '/../Views/errors/404.php';
    }
}
}

```

O arquivo Auth.php centraliza todo o controle de acesso do sistema. Seus métodos estáticos verificam o estado da sessão e determinam o cargo do usuário a partir das flags

armazenadas. Os métodos `requireLogin()`, `requireRedator()`, `requireRevisor()` e `requireAdmin()` são chamados no início de cada método protegido dos controllers, redirecionando automaticamente usuários não autorizados.

Código 2 — `app/Core/Auth.php`

```
<?php
namespace App\Core;

class Auth
{
    public static function start(): void
    {
        if (session_status() === PHP_SESSION_NONE) session_start();
    }

    public static function check(): bool
    {
        self::start();
        return !empty($_SESSION['usuario_logado']) &&
            !empty($_SESSION['usuario_id']);
    }

    public static function id(): ?int
    {
        self::start();
        return isset($_SESSION['usuario_id']) ? (int)$_SESSION['usuario_id'] :
null;
    }

    public static function cargo(): string
    {
        self::start();
        return $_SESSION['usuario_cargo'] ?? 'leitor';
    }

    public static function isAdmin(): bool { return self::cargo() ===
'administrador'; }
    public static function isRevisor(): bool { return in_array(self::cargo(),
['revisor','administrador']); }
    public static function isRedator(): bool { return in_array(self::cargo(),
['redator','revisor','administrador']); }

    public static function requireLogin(): void
    {
        if (!self::check()) { header('Location: '.url('/login')); exit; }
    }
    public static function requireRedator(): void
    {
        self::requireLogin();
        if (!self::isRedator()) { header('Location: '.url('/')); exit; }
    }
    public static function requireRevisor(): void
    {
        self::requireLogin();
        if (!self::isRevisor()) { header('Location: '.url('/')); exit; }
    }
    public static function requireAdmin(): void
```

```

    {
        self::requireLogin();
        if (!self::isAdmin()) { header('Location: '.url('/')); exit; }
    }
}

```

4.2 Banco de Dados — Database.php e schema.sql

A classe Database.php implementa o padrão de projeto Singleton para a conexão com o banco de dados. Isso significa que apenas uma instância de PDO é criada durante o ciclo de vida de uma requisição, sendo reutilizada em todas as consultas. A conexão é configurada com `ERRMODE_EXCEPTION` (lança exceções em erros SQL), `FETCH_ASSOC` (retorna resultados como arrays associativos) e `EMULATE_PREPARES` desativado (usa prepared statements reais do MySQL, aumentando a segurança).

Código 3 — app/Core/Database.php

```

<?php
namespace App\Core;

use PDO;
use PDOException;

class Database
{
    private static ?PDO $connection = null;

    public static function getConnection(): PDO
    {
        if (self::$connection === null) {
            $config = require __DIR__ . '/../../config/database.php';
            $dsn =
                "mysql:host={$config['host']};dbname={$config['database']};charset=utf8mb4";
            try {
                self::$connection = new PDO($dsn, $config['user'],
                    $config['password'], [
                        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                        PDO::ATTR_EMULATE_PREPARES => false,
                    ]);
            } catch (PDOException $e) {
                if (defined('APP_ENV') && APP_ENV === 'local')
                    die('Erro de conexão: ' . $e->getMessage());
                die('Erro ao conectar ao banco de dados.');
```

O arquivo schema.sql define a estrutura completa do banco de dados. Cada tabela foi projetada para refletir as entidades do domínio do sistema. O uso de campos ENUM garante

que apenas valores válidos sejam inseridos para status e ações. As chaves estrangeiras asseguram a integridade referencial: não é possível criar uma notícia sem um redator cadastrado, nem uma revisão sem notícia e revisor existentes.

Código 4 — database/schema.sql

```
CREATE DATABASE IF NOT EXISTS jornal_atlasdb;
USE jornal_atlasdb;

CREATE TABLE Usuario (
  id          INT PRIMARY KEY AUTO_INCREMENT,
  nome        VARCHAR(100) NOT NULL,
  nascimento  DATE NOT NULL,
  formacao    VARCHAR(150) NOT NULL,
  assinatura  VARCHAR(150) NOT NULL,
  email       VARCHAR(150) NOT NULL UNIQUE,
  senha       VARCHAR(255) NOT NULL,
  foto        VARCHAR(255) NULL,
  podeRedigir BOOLEAN NOT NULL DEFAULT FALSE,
  podeRevisar BOOLEAN NOT NULL DEFAULT FALSE,
  isAdmin     BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE TABLE Noticia (
  id          INT PRIMARY KEY AUTO_INCREMENT,
  redator_id  INT NOT NULL,
  titulo      VARCHAR(200) NOT NULL,
  resumo      VARCHAR(500) NOT NULL,
  conteudo    TEXT NOT NULL,
  imagem      VARCHAR(255) NULL,
  categoria   VARCHAR(50) NOT NULL,
  secao       ENUM('hero', 'nacional', 'internacional') NOT NULL,
  status      ENUM('RASCUNHO', 'EM_ANALISE', 'APROVADA', 'ARQUIVADA', 'REJEITADA')
              NOT NULL DEFAULT 'RASCUNHO',
  dataCriacao DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  dataPublicacao DATETIME NULL,
  dataEdicao   DATETIME NULL,
  FOREIGN KEY (redator_id) REFERENCES Usuario(id)
);

CREATE TABLE Revisao (
  id          INT PRIMARY KEY AUTO_INCREMENT,
  revisor_id  INT NOT NULL,
  noticia_id  INT NOT NULL,
  acaoRealizada ENUM('APROVAR', 'REJEITAR', 'ARQUIVAR', 'SOLICITAR_CORRECAO')
              NOT NULL,
  dataRevisao DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  observacao  TEXT,
  FOREIGN KEY (revisor_id) REFERENCES Usuario(id),
  FOREIGN KEY (noticia_id) REFERENCES Noticia(id)
);

CREATE TABLE SolicitacaoCargo (
  id          INT PRIMARY KEY AUTO_INCREMENT,
  usuario_id  INT NOT NULL,
  admin_id    INT NULL,
  cargo       ENUM('REDATOR', 'REVISOR') NOT NULL DEFAULT 'REDATOR',
```

```

        status          ENUM('EM_ANALISE','APROVADA','REJEITADA') NOT NULL DEFAULT
        'EM_ANALISE',
        dataSolicitacao  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
        dataResposta     DATETIME NULL,
        observacao       TEXT,
        FOREIGN KEY (usuario_id) REFERENCES Usuario(id),
        FOREIGN KEY (admin_id)  REFERENCES Usuario(id)
    );

```

4.3 Autenticação — AuthController.php

O AuthController gerencia os três fluxos de autenticação: login, cadastro e logout. No método `authenticate()`, o e-mail informado é buscado via `UsuarioRepository::findByEmail()` e a senha é validada com `password_verify()`, que compara o input com o hash bcrypt armazenado no banco. Em caso de sucesso, o ID de sessão é regenerado com `session_regenerate_id(true)`, prevenindo ataques de fixação de sessão, e os dados do usuário são gravados na sessão.

No método `register()`, são realizadas validações como formato de e-mail (`filter_var`), comprimento mínimo da senha (8 caracteres), confirmação de senha e unicidade do e-mail antes de persistir o novo usuário. O logout simplesmente destrói a sessão e redireciona para a página inicial.

Código 5 — app/Controllers/AuthController.php

```

<?php
namespace App\Controllers;

use App\Models\Usuario;
use App\Repositories\UsuarioRepository;

class AuthController
{
    public function login(): void
    {
        require __DIR__ . '/../Views/auth/login.php';
    }

    public function cadastro(): void
    {
        require __DIR__ . '/../Views/auth/cadastro.php';
    }

    public function authenticate(): void
    {
        $email = trim($_POST['email'] ?? '');
        $senha = $_POST['senha'] ?? '';
        $usuario = UsuarioRepository::findByEmail($email);
    }
}

```

```

        if (!$usuario || !password_verify($senha, $usuario->getSenha())) {
            $_SESSION['errors'] = ['E-mail ou senha inválidos.'];
            $_SESSION['old']     = ['email' => $email];
            header('Location: ' . url('/login')); exit;
        }

        session_regenerate_id(true);
        $_SESSION['usuario_logado'] = true;
        $_SESSION['usuario_id']     = $usuario->getId();
        $_SESSION['usuario_nome']   = $usuario->getNome();
        $_SESSION['usuario_email']  = $usuario->getEmail();
        $_SESSION['usuario_cargo']  = $usuario->getCargo();
        $_SESSION['usuario_foto']   = $usuario->getFoto();
        header('Location: ' . url('/')); exit;
    }

    public function register(): void
    {
        $nome      = trim($_POST['nome'] ?? '');
        $email     = trim($_POST['email'] ?? '');
        $senha     = $_POST['senha'] ?? '';
        $confirmar = $_POST['confirmar_senha'] ?? '';
        $errors    = [];

        if ($nome === '') $errors[] = 'O nome é obrigatório.';
        if ($email === '') $errors[] = 'O e-mail é obrigatório.';
        if (!filter_var($email, FILTER_VALIDATE_EMAIL)) $errors[] = 'E-mail inválido.';
        if (strlen($senha) < 8) $errors[] = 'A senha deve ter pelo menos 8 caracteres.';
        if ($senha !== $confirmar) $errors[] = 'As senhas não conferem.';
        if (UsuarioRepository::findByEmail($email)) $errors[] = 'E-mail já cadastrado.';

        if (!empty($errors)) {
            $_SESSION['errors'] = $errors;
            $_SESSION['old']    = ['nome' => $nome, 'email' => $email];
            header('Location: ' . url('/cadastro')); exit;
        }

        $usuario = new Usuario(0, $nome, new \DateTime('2000-01-01'), '', '', $email, $senha);
        UsuarioRepository::create($usuario);
        $_SESSION['sucesso'] = 'Conta criada com sucesso! Faça login.';
        header('Location: ' . url('/login')); exit;
    }

    public function logout(): void
    {
        session_start();
        session_destroy();
        header('Location: ' . url('/')); exit;
    }
}

```

4.4 Notícias — Noticia.php, NoticiaRepository.php e NoticiaController.php

A classe Noticia.php é o Model que representa uma notícia no sistema. Ela encapsula todos os atributos da entidade (título, resumo, conteúdo, imagem, categoria, seção, status e

datas) com getters e setters, seguindo o princípio de encapsulamento da orientação a objetos. O atributo status é tipado como StatusNoticia (um Enum PHP 8.1), garantindo que apenas valores válidos (RASCUNHO, EM_ANALISE, APROVADA, ARQUIVADA, REJEITADA) sejam atribuídos.

Código 6 — app/Models/Noticia.php

```
<?php
namespace App\Models;

use App\Core>StatusNoticia;
use DateTime;

class Noticia
{
    private int $id;
    private int $redator_id;
    private string $titulo;
    private string $resumo;
    private string $conteudo;
    private ?string $imagem;
    private string $categoria;
    private string $secao;
    private StatusNoticia $status;
    private DateTime $dataCriacao;
    private ?DateTime $dataPublicacao;
    private ?DateTime $dataEdicao;

    public function __construct(
        int $id, int $redator_id, string $titulo,
        string $resumo, string $conteudo, ?string $imagem,
        string $categoria, string $secao, StatusNoticia $status,
        DateTime $dataCriacao,
        ?DateTime $dataPublicacao = null, ?DateTime $dataEdicao = null
    ) {
        $this->id = $id;
        $this->redator_id = $redator_id;
        $this->titulo = $titulo;
        $this->resumo = $resumo;
        $this->conteudo = $conteudo;
        $this->imagem = $imagem;
        $this->categoria = $categoria;
        $this->secao = $secao;
        $this->status = $status;
        $this->dataCriacao = $dataCriacao;
        $this->dataPublicacao = $dataPublicacao;
        $this->dataEdicao = $dataEdicao;
    }

    // Getters e Setters
    public function getId(): int { return $this->id; }
    public function setId(int $id): void { $this->id = $id; }
    public function getRedatorId(): int { return $this->redator_id; }
    public function getTitulo(): string { return $this->titulo; }
    public function setTitulo(string $t): void { $this->titulo = $t; }
    public function getResumo(): string { return $this->resumo; }
    public function setResumo(string $r): void { $this->resumo = $r; }
```

```

    public function getConteudo(): string      { return $this->conteudo; }
    public function setConteudo(string $c): void { $this->conteudo = $c; }
    public function getImagem(): ?string      { return $this->imagem; }
    public function setImagem(?string $i): void { $this->imagem = $i; }
    public function getCategoria(): string    { return $this->categoria; }
    public function setCategoria(string $c): void { $this->categoria = $c; }
    public function getSecao(): string        { return $this->secao; }
    public function setSecao(string $s): void { $this->secao = $s; }
    public function getStatus(): StatusNoticia { return $this->status; }
    public function setStatus(StatusNoticia $s): void { $this->status = $s; }
    public function getDataCriacao(): DateTime { return $this->dataCriacao; }
>dataCriacao; }
    public function getDataPublicacao(): ?DateTime { return $this->dataPublicacao; }
>dataPublicacao; }
    public function setDataPublicacao(?DateTime $d): void { $this->dataPublicacao = $d; }
>dataPublicacao = $d; }
    public function getDataEdicao(): ?DateTime { return $this->dataEdicao; }
>dataEdicao; }
}

```

O `NoticiaRepository` concentra todas as consultas SQL relacionadas à tabela `Noticia`. Ele utiliza prepared statements em todas as operações de escrita e nas consultas parametrizadas, prevenindo SQL injection. O método `hydrate()` converte um array de dados brutos do banco em um objeto `Noticia` tipado, centralizando a lógica de mapeamento. O método `search()` implementa busca textual por meio de cláusulas `LIKE` aplicadas ao título, resumo e conteúdo das notícias aprovadas.

Código 7 — `app/Repositories/NoticiaRepository.php`

```

<?php
namespace App\Repositories;

use App\Core\Database;
use App\Core>StatusNoticia;
use App\Models\Noticia;
use DateTime;

class NoticiaRepository
{
    public static function find(int $id): ?Noticia
    {
        $stmt = Database::getConnection()->prepare(
            'SELECT * FROM Noticia WHERE id = :id LIMIT 1'
        );
        $stmt->execute([':id' => $id]);
        $row = $stmt->fetch();
        return $row ? self::hydrate($row) : null;
    }

    public static function approved(): array
    {
        $stmt = Database::getConnection()->query(
            'SELECT * FROM Noticia WHERE status = \'APROVADA\' ORDER BY
dataPublicacao DESC'

```

```

    );
    return self::hydrateMany($stmt->fetchAll());
}

public static function byRedator(int $redatorId): array
{
    $stmt = Database::getConnection()->prepare(
        'SELECT * FROM Noticia WHERE redator_id = :id ORDER BY dataCriacao
DESC'
    );
    $stmt->execute([':id' => $redatorId]);
    return self::hydrateMany($stmt->fetchAll());
}

public static function byCategoria(string $categoria): array
{
    $stmt = Database::getConnection()->prepare(
        'SELECT * FROM Noticia WHERE status = :status AND categoria = :cat
ORDER BY dataPublicacao DESC'
    );
    $stmt->execute([':status' => StatusNoticia::APROVADA->value, ':cat' =>
$categoria]);
    return self::hydrateMany($stmt->fetchAll());
}

public static function search(string $termo): array
{
    $t = '%' . $termo . '%';
    $stmt = Database::getConnection()->prepare(
        'SELECT * FROM Noticia WHERE status = :status
AND (titulo LIKE :t1 OR resumo LIKE :t2 OR conteudo LIKE :t3)
ORDER BY dataPublicacao DESC'
    );
    $stmt->execute([
        ':status' => StatusNoticia::APROVADA->value,
        ':t1' => $t, ':t2' => $t, ':t3' => $t,
    ]);
    return $stmt->fetchAll();
}

public static function create(Noticia $noticia): int
{
    $stmt = Database::getConnection()->prepare(
        'INSERT INTO Noticia
(redator_id,titulo,resumo,conteudo,imagem,categoria,secao,status)
VALUES
(:redator_id,:titulo,:resumo,:conteudo,:imagem,:categoria,:secao,:status)'
    );
    $stmt->execute([
        ':redator_id' => $noticia->getRedatorId(),
        ':titulo' => $noticia->getTitulo(),
        ':resumo' => $noticia->getResumo(),
        ':conteudo' => $noticia->getConteudo(),
        ':imagem' => $noticia->getImagem(),
        ':categoria' => $noticia->getCategoria(),
        ':secao' => $noticia->getSecao(),
        ':status' => $noticia->getStatus()->value,
    ]);
    return (int) Database::getConnection()->lastInsertId();
}

public static function update(Noticia $noticia): bool
{

```

```

        $sql = 'UPDATE Noticia SET titulo=:titulo, resumo=:resumo,
            conteudo=:conteudo, imagem=:imagem, categoria=:categoria,
            secao=:secao, status=:status, dataEdicao=NOW()';
        $params = [
            ':id' => $noticia->getId(),
            ':titulo' => $noticia->getTitulo(),
            ':resumo' => $noticia->getResumo(),
            ':conteudo' => $noticia->getConteudo(),
            ':imagem' => $noticia->getImagem(),
            ':categoria' => $noticia->getCategoria(),
            ':secao' => $noticia->getSecao(),
            ':status' => $noticia->getStatus()->value,
        ];
        if ($noticia->getDataPublicacao() != null) {
            $sql .= ', dataPublicacao = :dataPublicacao';
            $params[':dataPublicacao'] = $noticia->getDataPublicacao()-
>format('Y-m-d H:i:s');
        }
        $sql .= ' WHERE id = :id';
        return Database::getConnection()->prepare($sql)->execute($params);
    }

    public static function delete(int $id): bool
    {
        $stmt = Database::getConnection()->prepare('DELETE FROM Noticia WHERE id
= :id');
        return $stmt->execute([':id' => $id]);
    }

    private static function hydrate(array $row): Noticia
    {
        return new Noticia(
            (int)$row['id'], (int)$row['redator_id'],
            $row['titulo'], $row['resumo'], $row['conteudo'],
            $row['imagem'] ?: null, $row['categoria'], $row['secao'],
            StatusNoticia::from($row['status']),
            new DateTime($row['dataCriacao']),
            $row['dataPublicacao'] ? new DateTime($row['dataPublicacao']) :
null,
            $row['dataEdicao'] ? new DateTime($row['dataEdicao']) : null
        );
    }

    private static function hydrateMany(array $rows): array
    {
        return array_map([self::class, 'hydrate'], $rows);
    }
}

```

O `NoticiaController` é o componente mais extenso do sistema, gerenciando o ciclo de vida completo de uma notícia. O método `store()` recebe os dados do formulário, valida campos obrigatórios, processa o upload de imagem (verificando a extensão do arquivo e salvando em `public/assets/img/`) e persiste a notícia. O status inicial é `RASCUNHO` ou `EM_ANALISE`, dependendo da ação selecionada pelo redator. O método `update()` verifica se o usuário tem permissão para editar e, ao salvar uma notícia aprovada/rejeitada/arquivada, retorna

automaticamente seu status para EM_ANALISE. O método `excluirRascunho()` só permite exclusão de notícias ainda em estado de rascunho pelo próprio redator.

Código 8 — `app/Controllers/NoticiaController.php`

```
<?php
namespace App\Controllers;

use App\Core\Auth;
use App\Core>StatusNoticia;
use App\Models\Noticia;
use App\Repositories\NoticiaRepository;

class NoticiaController
{
    public function create(): void
    {
        Auth::requireRedator();
        require __DIR__ . '/../Views/noticia/create.php';
    }

    public function minhas(): void
    {
        Auth::requireRedator();
        $noticias = NoticiaRepository::byRedator(Auth::id());
        require __DIR__ . '/../Views/noticia/minhas.php';
    }

    public function store(): void
    {
        Auth::requireRedator();
        $titulo = trim($_POST['titulo'] ?? '');
        $categoria = trim($_POST['categoria'] ?? '');
        $secao = trim($_POST['secao'] ?? '');
        $resumo = trim($_POST['resumo'] ?? '');
        $conteudo = trim($_POST['conteudo'] ?? '');
        $errors = [];

        if ($titulo === '') $errors[] = 'O título é obrigatório.';
        if ($categoria === '') $errors[] = 'A categoria é obrigatória.';
        if ($secao === '') $errors[] = 'A seção é obrigatória.';
        if ($resumo === '') $errors[] = 'O resumo é obrigatório.';
        if ($conteudo === '') $errors[] = 'O conteúdo é obrigatório.';

        $imagem = null;
        if (isset($_FILES['imagem']) && $_FILES['imagem']['error'] ===
        UPLOAD_ERR_OK) {
            $ext = strtolower(pathinfo($_FILES['imagem']['name'],
        PATHINFO_EXTENSION));
            $allowed = ['jpg', 'jpeg', 'png', 'webp'];
            if (!in_array($ext, $allowed)) {
                $errors[] = 'Formato de imagem inválido.';
            } else {
                $nome = 'noticia_' . time() . '_' . mt_rand(1000, 9999) . '.' . $ext;
                $dest = __DIR__ . '/../public/assets/img/' . $nome;
                if (move_uploaded_file($_FILES['imagem']['tmp_name'], $dest))
                    $imagem = $nome;
                else
                    $errors[] = 'Erro ao fazer upload da imagem.';
            }
        }
    }
}
```

```

    }

    if (!empty($errors)) {
        $_SESSION['errors'] = $errors;
        header('Location: '.url('/noticia/nova')); exit;
    }

    $acao = $_POST['acao'] ?? 'salvar';
    $status = ($acao === 'enviar') ? StatusNoticia::ANALISE :
StatusNoticia::RASCUNHO;

    NoticiaRepository::create(new Noticia(
        0, Auth::id(), $titulo, $resumo, $conteudo,
        $imagem, $categoria, $secao, $status, new \DateTime()
    ));

    $_SESSION['sucesso'] = $acao === 'enviar'
        ? 'Notícia enviada para revisão!' : 'Rascunho salvo!';
    header('Location: '.url('/')); exit;
}

public function update(string $id): void
{
    Auth::requireRedator();
    $noticia = NoticiaRepository::find((int)$id);
    if (!$noticia) { header('Location: '.url('/noticia/minhas')); exit; }
    if (!Auth::isAdmin() && !Auth::isRevisor()
        && $noticia->getRedatorId() !== Auth::id()) {
        header('Location: '.url('/noticia/minhas')); exit;
    }

    // [validação e upload idênticos ao store()]

    $acao = $_POST['acao'] ?? 'salvar';
    if ($acao === 'reenviar'
        || in_array($noticia->getStatus(), [
            StatusNoticia::APROVADA,
            StatusNoticia::REJEITADA,
            StatusNoticia::ARQUIVADA
        ])) {
        $noticia->setStatus(StatusNoticia::ANALISE);
    }
    NoticiaRepository::update($noticia);
    header('Location: '.url('/noticia/minhas')); exit;
}

public function excluirRascunho(string $id): void
{
    Auth::requireRedator();
    $noticia = NoticiaRepository::find((int)$id);
    if (!$noticia || $noticia->getRedatorId() !== Auth::id()
        || $noticia->getStatus() !== StatusNoticia::RASCUNHO) {
        header('Location: '.url('/noticia/minhas')); exit;
    }
    NoticiaRepository::delete((int)$id);
    $_SESSION['sucesso'] = 'Rascunho excluído com sucesso!';
    header('Location: '.url('/noticia/minhas')); exit;
}
}

```

4.5 Revisão Editorial — RevisaoController.php

O `RevisaoController` implementa o fluxo de revisão editorial com três ações: `aprovar()`, `rejeitar()` e `arquivar()`. Em todas elas, o sistema primeiro verifica se o usuário tem permissão de revisor e, em seguida, aplica a regra de negócio mais importante do fluxo editorial: um redator não pode revisar o próprio artigo. Essa restrição é implementada comparando o ID do redator da notícia com o ID do usuário autenticado — a exceção é o administrador, que pode realizar qualquer ação.

Ao aprovar uma notícia, o sistema registra a revisão na tabela `Revisao` com a ação `APROVAR`, atualiza o status da notícia para `APROVADA` e define a `dataPublicacao` com o timestamp atual, tornando a notícia visível publicamente. Ao rejeitar, o status vai para `REJEITADA` e o redator pode visualizar a observação deixada pelo revisor para corrigir e reenviar o conteúdo.

Código 9 — `app/Controllers/RevisaoController.php`

```
<?php
namespace App\Controllers;

use App\Core\Auth;
use App\Core\AcaoRevisao;
use App\Core>StatusNoticia;
use App\Models\Revisao;
use App\Repositories\RevisaoRepository;
use App\Repositories\NoticiaRepository;

class RevisaoController
{
    public function index(): void
    {
        Auth::requireLogin();
        $userId = Auth::id();
        $isAdmin = Auth::isAdmin();
        $isRevisor = Auth::isRevisor();

        if ($isAdmin || $isRevisor) {
            $noticias = RevisaoRepository::pendentesEHistorico($userId);
            $podeAprovar = false;
            $noticiaSelecionada = null;
            $noticiaId = $_GET['noticia'] ?? null;
            if ($noticiaId) {
                $noticiaSelecionada = NoticiaRepository::find((int)$noticiaId);
                if ($noticiaSelecionada) {
                    $podeAprovar = $isAdmin
                        || ($noticiaSelecionada->getRedatorId() !== $userId);
                }
            }
        }
    }
}
```

```

        require __DIR__ . '/../Views/revisao/index.php';
    } else {
        $minhasNoticias = NoticiaRepository::byRedator($userId);
        require __DIR__ . '/../Views/revisao/minhas.php';
    }
}

public function aprovar(string $id): void
{
    Auth::requireRevisor();
    $noticia = NoticiaRepository::find((int)$id);
    $revisorId = Auth::id();

    if ($noticia->getRedatorId() === $revisorId && !Auth::isAdmin()) {
        $_SESSION['erro'] = 'Você não pode aprovar seu próprio artigo.';
        header('Location: '.url('/revisao?noticia='.$id)); exit;
    }

    $obs = trim($_POST['observacao'] ?? '');
    RevisaoRepository::create(
        new Revisao(0, $revisorId, (int)$id, AcaoRevisao::APROVAR, new
\DateTime(), $obs ?: null)
    );
    $noticia->setStatus(StatusNoticia::APROVADA);
    $noticia->setDataPublicacao(new \DateTime());
    NoticiaRepository::update($noticia);
    $_SESSION['sucesso'] = 'Notícia aprovada com sucesso!';
    header('Location: '.url('/revisao')); exit;
}

public function rejeitar(string $id): void
{
    Auth::requireRevisor();
    $noticia = NoticiaRepository::find((int)$id);
    $revisorId = Auth::id();

    if ($noticia->getRedatorId() === $revisorId && !Auth::isAdmin()) {
        $_SESSION['erro'] = 'Você não pode rejeitar seu próprio artigo.';
        header('Location: '.url('/revisao?noticia='.$id)); exit;
    }

    $obs = trim($_POST['observacao'] ?? '');
    RevisaoRepository::create(
        new Revisao(0, $revisorId, (int)$id, AcaoRevisao::REJEITAR, new
\DateTime(), $obs ?: null)
    );
    $noticia->setStatus(StatusNoticia::REJEITADA);
    NoticiaRepository::update($noticia);
    $_SESSION['sucesso'] = 'Notícia rejeitada.';
    header('Location: '.url('/revisao')); exit;
}

public function arquivar(string $id): void
{
    Auth::requireRevisor();
    $noticia = NoticiaRepository::find((int)$id);
    $revisorId = Auth::id();

    if ($noticia->getRedatorId() === $revisorId && !Auth::isAdmin()) {
        $_SESSION['erro'] = 'Você não pode arquivar seu próprio artigo.';
        header('Location: '.url('/revisao?noticia='.$id)); exit;
    }
}

```



```

        $obs = trim($_POST['observacao'] ?? '');
        RevisaoRepository::create(
            new Revisao(0, $revisorId, (int)$id, AcaoRevisao::ARQUIVAR, new
\DateTime(), $obs ?? null)
        );
        $noticia->setStatus(StatusNoticia::ARQUIVADA);
        NoticiaRepository::update($noticia);
        $_SESSION['sucesso'] = 'Notícia arquivada.';
        header('Location: '.url('/revisao')); exit;
    }
}

```

4.6 Solicitação de Cargo — SolicitacaoController.php

O SolicitacaoController gerencia o processo pelo qual um usuário comum solicita o cargo de Redator ou Revisor. Todas as ações são restritas ao administrador via `Auth::requireAdmin()`. O método `index()` exibe a fila de solicitações pendentes com estatísticas.

O método `aprovar()` é o mais relevante: além de atualizar o status da solicitação para APROVADA no banco, ele chama `UsuarioRepository::updatePermissoes()` para ativar as flags correspondentes (`podeRedigir` ou `podeRevisar`) diretamente no registro do usuário. Essa operação é atômica — a permissão é concedida imediatamente, sem necessidade de nenhuma ação adicional. O método `rejeitar()` apenas atualiza o status para REJEITADA, mantendo as permissões do usuário inalteradas.

Código 10 — app/Controllers/SolicitacaoController.php

```

<?php
namespace App\Controllers;

use App\Core\Auth;
use App\Core>StatusSolicitacao;
use App\Repositories\SolicitacaoCargoRepository;
use App\Repositories\UsuarioRepository;

class SolicitacaoController
{
    public function index(): void
    {
        Auth::requireAdmin();
        $solicitacoes = SolicitacaoCargoRepository::allWithUser();
        $stats = SolicitacaoCargoRepository::stats();
        $selecionada = null;
        $selecionadaUser = null;
        $solId = $_GET['id'] ?? null;
        if ($solId) {
            $selecionada = SolicitacaoCargoRepository::find((int)$solId);
            if ($selecionada)

```

```

        $selecionadaUser = UsuarioRepository::find($selecionada->getUsuarioId());
    }
    require __DIR__ . '/../Views/solicitacao/index.php';
}

public function aprovar(string $id): void
{
    Auth::requireAdmin();
    $sol = SolicitacaoCargoRepository::find((int)$id);
    if (!$sol) { header('Location: '.url('/solicitacoes')); exit; }

    $obs      = trim($_POST['observacao'] ?? '');
    $adminId = Auth::id();
    SolicitacaoCargoRepository::responder(
        (int)$id, StatusSolicitacao::APROVADA, $adminId, $obs ?: null
    );

    $usuario = UsuarioRepository::find($sol->getUsuarioId());
    if ($usuario) {
        if ($sol->getCargo()->value === 'REDATOR')
            UsuarioRepository::updatePermissoes(
                $usuario->getId(), true, $usuario->getPodeRevisar(),
                $usuario->getIsAdmin()
            );
        elseif ($sol->getCargo()->value === 'REVISOR')
            UsuarioRepository::updatePermissoes(
                $usuario->getId(), $usuario->getPodeRedigir(), true,
                $usuario->getIsAdmin()
            );
    }
    $_SESSION['sucesso'] = 'Solicitação aprovada! Permissões atualizadas.';
    header('Location: '.url('/solicitacoes')); exit;
}

public function rejeitar(string $id): void
{
    Auth::requireAdmin();
    $sol = SolicitacaoCargoRepository::find((int)$id);
    if (!$sol) { header('Location: '.url('/solicitacoes')); exit; }

    $obs      = trim($_POST['observacao'] ?? '');
    $adminId = Auth::id();
    SolicitacaoCargoRepository::responder(
        (int)$id, StatusSolicitacao::REJEITADA, $adminId, $obs ?: null
    );
    $_SESSION['sucesso'] = 'Solicitação rejeitada.';
    header('Location: '.url('/solicitacoes')); exit;
}
}

```

5 CONSIDERAÇÕES FINAIS

O desenvolvimento do Atlas Jornal possibilitou a aplicação prática de conceitos fundamentais do desenvolvimento web, como o padrão MVC, segurança em autenticação e acesso, modelagem relacional de banco de dados e controle de fluxo em sistemas com múltiplos

perfis de usuário. A implementação sem frameworks permitiu que os desenvolvedores compreendessem em profundidade o funcionamento interno de cada camada da aplicação.

O sistema resultante atende aos requisitos propostos para a disciplina, apresentando um portal jornalístico funcional com fluxo editorial completo, controle granular de permissões e interface responsiva. Como trabalho futuro, identificam-se oportunidades de melhoria como a implementação de notificações por e-mail, paginação de resultados, suporte a múltiplas imagens por notícia e a adoção de um framework PHP para maior produtividade e padronização do código.

REFERÊNCIAS

PHP GROUP. PHP Manual. Disponível em: https://www.php.net/manual/pt_BR/. Acesso em: jun. 2026.

WELLING, Luke; THOMSON, Laura. **PHP e MySQL: Desenvolvimento Web**. 5. ed. Rio de Janeiro: Alta Books, 2018. Acesso em: jun. 2026.

ORACLE CORPORATION. MySQL 8.0 Reference Manual. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>. Acesso em: jun. 2026.

DALL'OGGIO, Pablo. **PHP: Programando com Orientação a Objetos**. 4. ed. São Paulo: Novatec, 2021. Acesso em: jun. 2026.

GAMMA, E. et al. Padrões de Projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. Acesso em: jun. 2026.